Agentic Summarizer Walkthrough — Executive + Technical

Author: Jibril Anifowoshe — September 2025

Purpose

The Agentic Summarizer prototype demonstrates a local, agentic AI system built using Ollama and a minimal LangChain wrapper for tools. It ingests online articles, produces concise 5-bullet executive summaries, and can compare multiple sources.

What It Delivers

1. Single URL summarization into 5 executive-style bullets.

2. Batch (3-URL) summarization with structured comparison (themes, differences, unique insights, takeaway).

3. Export to Markdown and PDF with professional signature footer.

4. Interactive Jupyter notebook UI with tabbed controls.

Architecture Overview

The summarizer follows a modular architecture:

- **Cell 1:** Bootstrap and health check (dependencies + Ollama server).

- **Cell 2:** Tools (`fetch_url`, `save_note`).

- **Cell 3:** Summarizer (direct Ollama API call with safety stops).

- **Cell 4:** Tabbed UI (Single URL vs. 3 URLs + Compare).

- **src/**: Source code for tools, summarizer, and UI modules.

- **examples/**: Sample `.md` and `.pdf` outputs.

- **tests/**: Unit-style checks for fetch, save, and summarize.

Cell 1 — Bootstrap + Health Check

Installs dependencies, imports core libraries, and ensures Ollama is running locally on `http://127.0.0.1:11434`.

```
# Install & check packages
_pip_install([
    "requests", "beautifulsoup4", "readability-lxml",
    "ipywidgets", "reportlab", "langchain>=0.2"
])

# Health check
def _ollama_running(...): ...
```

Cell 2 — Tools

Defines two tools:

- `fetch_url`: Fetches a web page, extracts readable text, trims to safe length.

- `save_note`: Saves summaries as `.md` files with a signature footer.

```
@tool
def fetch_url(url: str) -> str:
    # fetches, cleans, trims to 20k chars
    return text[:20000]

@tool
def save_note(filename: str, content: str) -> str:
    # saves with footer
    return f"Saved notes to {filename}"
```

Cell 3 — Summarizer

Calls Ollama directly (via `/api/generate`) with a system prompt that enforces exactly 5 bullets. Uses stop tokens to prevent run-on output.

```
MODEL = "llama3.2:3b"
SYSTEM = """You are a concise research agent.
Return EXACTLY 5 bullets as a NUMBERED LIST..."""

def _ollama_generate(prompt: str): ...
def summarize_text(article_text: str, max_chars=2800): ...
```

Cell 4 — Tabbed UI

Provides two tabs in Jupyter: Single URL summarizer and 3 URLs + Compare. Saves results in `.md` and optionally `.pdf`.

```
tabs = w.Tab(children=[single_tab, compare_tab])
tabs.set_title(0, "Single URL")
tabs.set_title(1, "3 URLs + Compare")
display(tabs)
```

Libraries Used

- **requests**: HTTP fetch for web pages and Ollama API calls.

- **beautifulsoup4**: Parse and clean article text.

- **readability-lxml**: Extracts main content from web pages.

- **ipywidgets**: Interactive UI in Jupyter (text boxes, tabs, buttons).

- **reportlab**: Generates PDF exports.

- **langchain.tools**: Provides the `@tool` decorator for clean integration.

Example Outputs

Sample results are stored under `examples/`:

- **test_single.md**: Single URL summary.

- **test_batch_comparison.md**: 3-URL batch comparison.

```
### Here's the summary of https://example.com
1. Point A
2. Point B
3. Point C
4. Point D
5. Point E
```

Tests

The `tests/` folder includes unit checks:

- `test_fetch.py`: Ensures URLs return non-empty cleaned text.

- `test_save.py`: Ensures `.md` file creation with footer.

- `test_summarize.py`: Ensures summarizer returns 5 bullets.

Future Enhancements

In a perfect world, with access to paid APIs:

- **OpenAI GPT-4o**: Higher accuracy, fewer truncations.

- **Anthropic Claude**: Strong long-context summarization.

- **Hybrid Agents**: Combining search + summarization for live analysis.

Closing Note

This project is a working demo of an agentic summarizer, local, transparent, and extendable. It is both a teaching artifact and a foundation for enterprise-grade summarization pipelines.